CS 4530: Fundamentals of Software Engineering
Module 12: Continuous Development

Adeel Bhutta and Mitch Wand

Khoury College of Computer Sciences

© 2026 Released under the CC BY-SA license

Learning objectives for this lesson

- By the end of this lesson, you should be able to…
  - Describe how continuous integration helps to catch errors sooner in the software lifecycle
  - Describe strategies for performing quality-assurance on software as and after it is delivered
  - Understand how continuous delivery can work with or without TDD (test driven development) as a quality assurance strategy

Review: The Agile Model Reduces Risk by Embracing Change (~2000)

- The Waterfall philosophy:
  - "The project is too large and complex, and it will take months (or years!) to plan, so once we come up with the plan, that plan can not change"
  - Reduce risk by proceeding in stages
- The Agile philosophy:
  - The project is too large and complex, it is unlikely that we will know exactly what we need right now, and to some extent, we are inventing something new. We think that as we make it, we will figure it out as we go”
  - Reduce risk by limiting time on any one stage; then reassess. (“time-boxing”)
  - Reduce risk through automated testing

Agile relies on a variety of quality-assurance processes

- What are the costs & benefits of each of these?
  - unit testing/TDD
  - code review
  - integration tests (as in module 10)
  - continuous integration
  - continuous deployment (A/B, canaries, etc.)
- How is each automatable?
- How does each address non-functional quality attributes?
- How should these be combined in an organization's software development process?

Example: Some bugs slip through testing, even in highly-regulated industries

Aviation

After Alaska Airlines planes bump runway while taking off from Seattle, a scramble to 'pull the plug'

By Dominic Gates, The Seattle Times
Updated: February 20, 2023
Published: February 20, 2023

"That morning, a software bug in an update to the DynamicSource tool caused it to provide seriously undervalued weights for the airplanes.

The Alaska 737 captain said the data was on the order of 20,000 to 30,000 pounds light. With the total weight of those jets at 150,000 to 170,000 pounds, the error was enough to skew the engine thrust and speed settings.

Both planes headed down the runway with less power and at lower speed than they should have. And with the jets judged lighter than they actually were, the pilots rotated too early

Both the Max 9 and 737-900ER have long passenger cabins, which makes them more vulnerable to a tail strike when the nose comes up too soon." …

… "A quick interim fix proved easy: When operations staff turned off the automatic uplink of the data to the aircraft and switched to manual requests "we didn't have the bug anymore."

Peyton said his team also checked the integrity of the calculation itself before lifting the stoppage. All that was accomplished in 20 minutes.

The software code was permanently repaired about five hours later.

Peyton added that even though the update to the DynamicSource software had been tested over an extended period, the bug was missed because it only presented when many aircraft at the same time were using the system.

Subsequently, a test of the software under high demand was developed."



Photo: saiters_photography (IG, different plane/airpot)

https://www.adn.com/alaska-news/aviation/2023/02/20/after-alaska-airlines-planes-bump-runway-a-scramble-to-pull-the-plug/

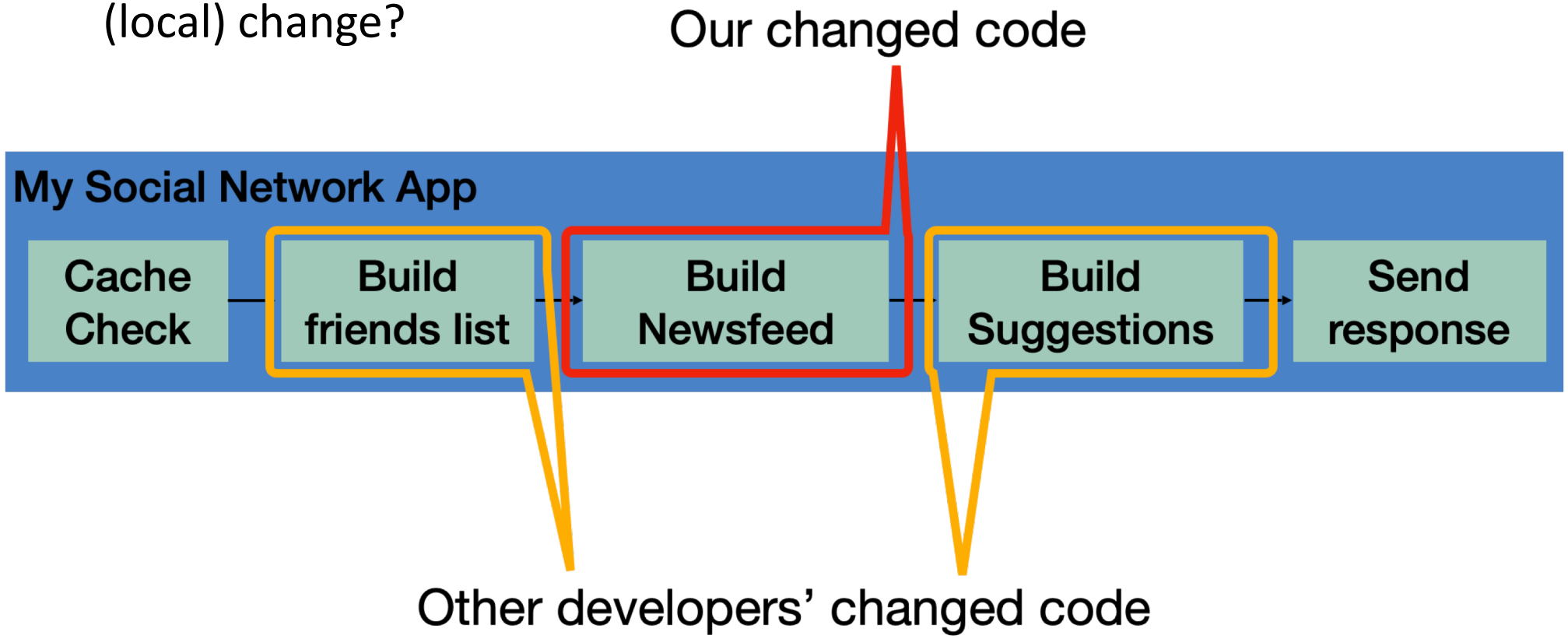
CI/CD practices improve code quality and dev velocity

- Continuous integration: use automated systems to perform and monitor frequent integrations with entire codebase, running integration-scale tests
- Continuous delivery: use automated systems to perform frequent, controlled delivery of product (often to a small fraction of the user base), with automated monitoring to detect remaining defects quickly.

12.1: Continuous Integration

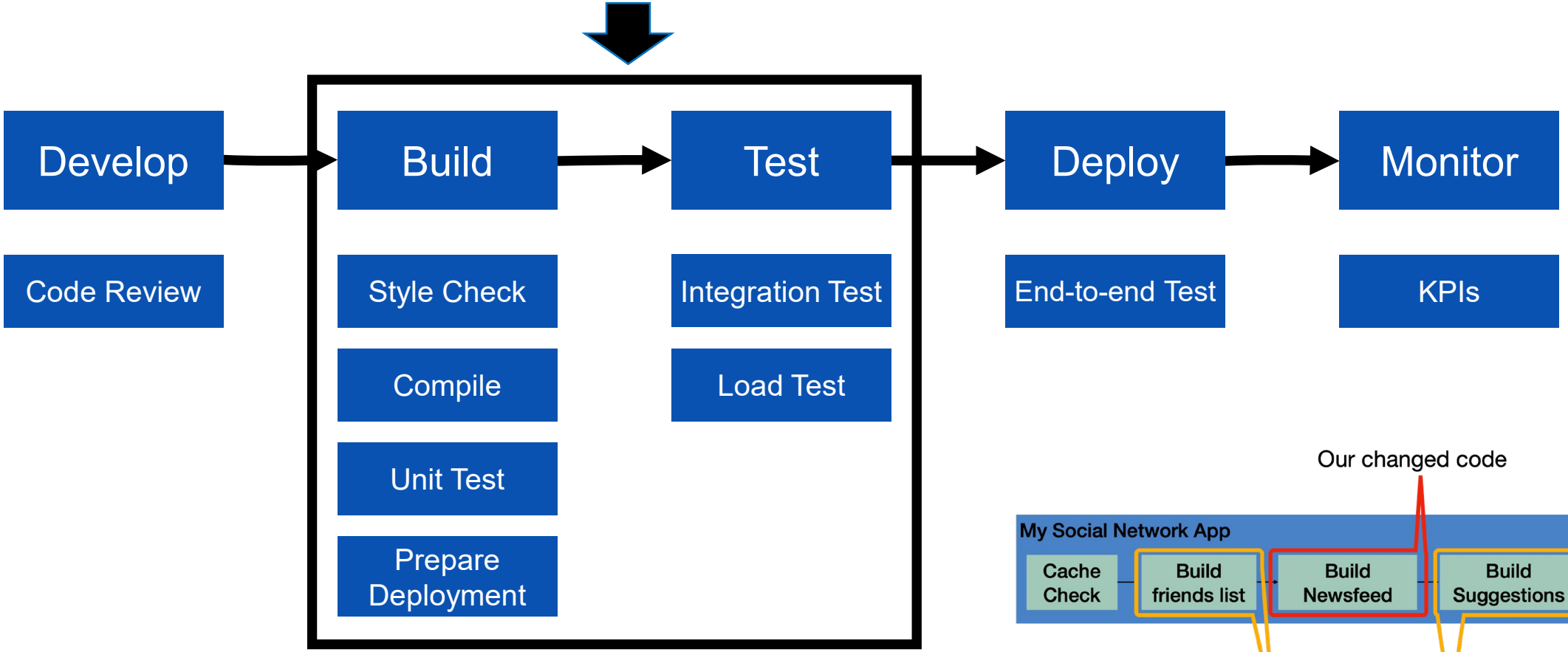
Continuous Integration (CI) provides global feedback on local changes

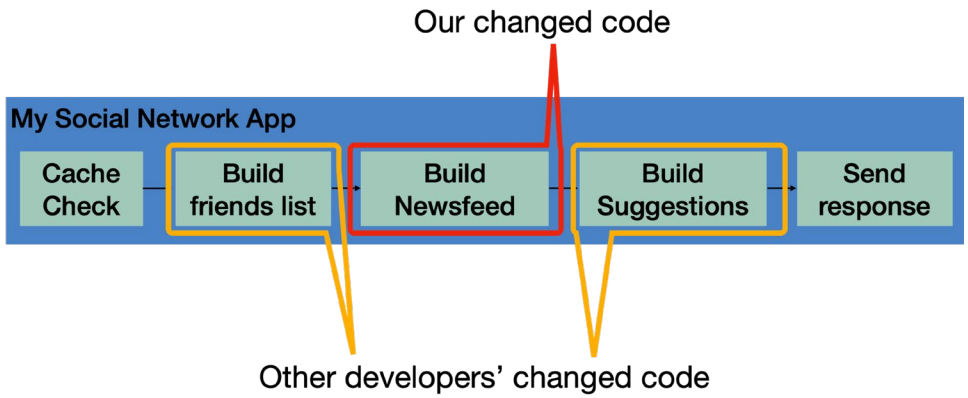
- Given: Our systems involve many components, some of which might even be in different version control repositories
- Consider: How does a developer get feedback on their (local) change?



A CI process is a software pipeline

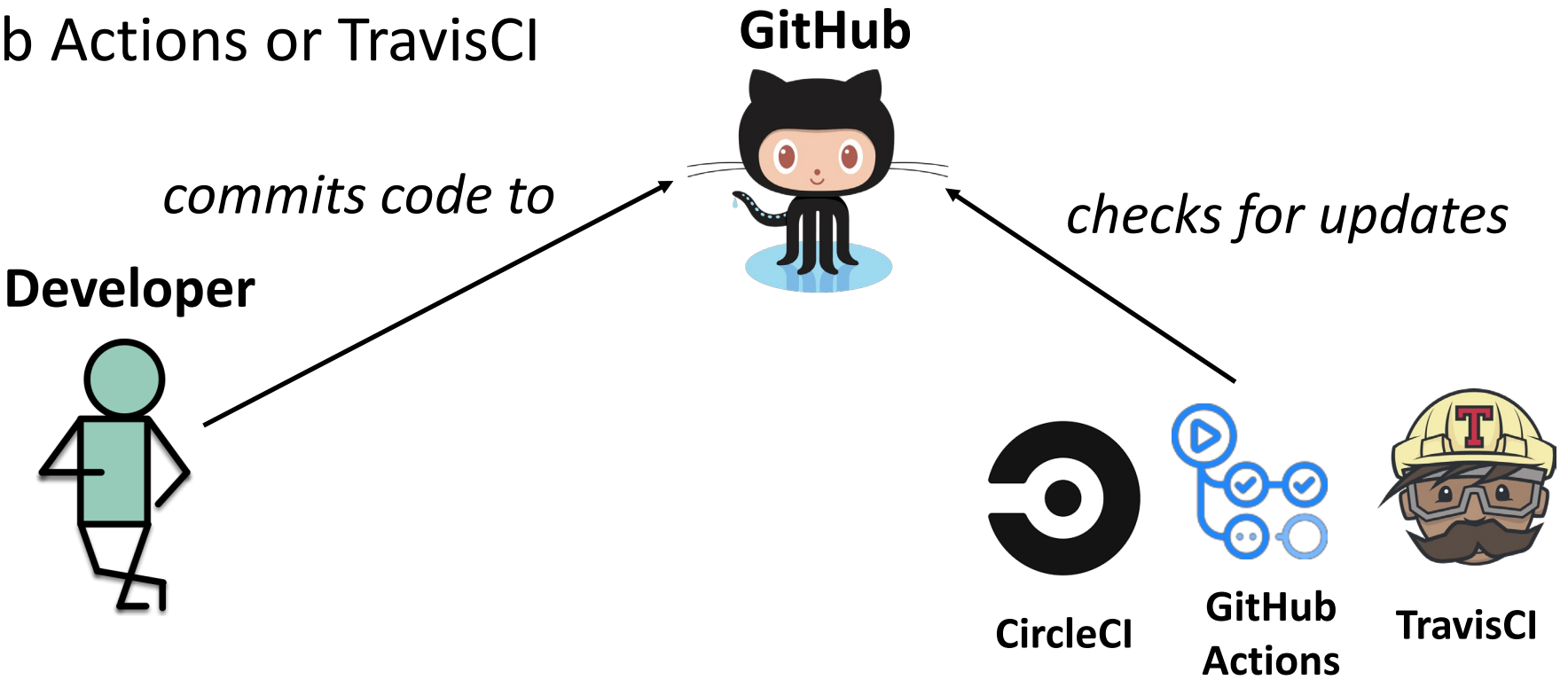
Automate this centrally, provide a central record of results





CI may be triggered by commits, pull requests, or other actions

Example: Small scale CI, with a service like CircleCI, GitHub Actions or TravisCI

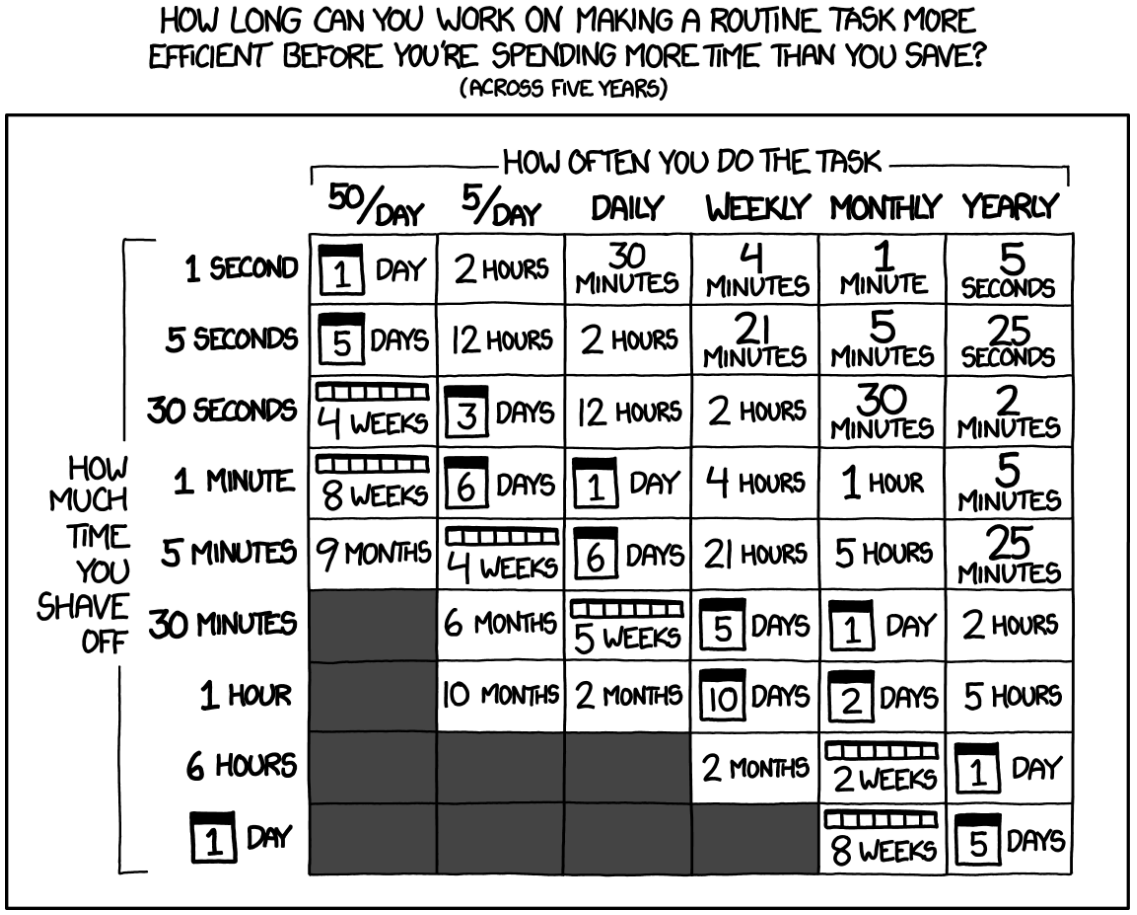


Runs build for each commit

Automating Feedback Loops is Powerful

Consider tasks that are done by *dozens* of developers (e.g. testing/deployment)

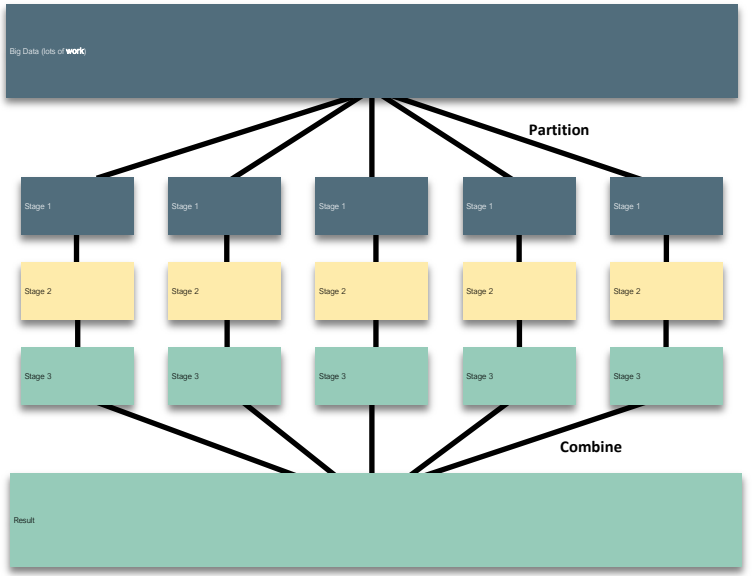
- If there is something that takes you 30 minutes to do every day, you will save 5 weeks of time over a 5-year period if you could automate it.
- Image that for 10k developers.



© Randal Munroe/xkcd, licensed CC-BY-SA
https://xkcd.com/1205/

Continuous Integration Pipeline is Highly Configurable

- Do we integrate changes immediately, or do pre-commit testing?
- Which tests do we run when we integrate? Example: run a short test daily (or often) or maybe on every commit and more comprehensive tests less often
- When do we do code review?
- A typical pipeline requires you to run tests against inputs (preferably in parallel) and records results and performance in central db.

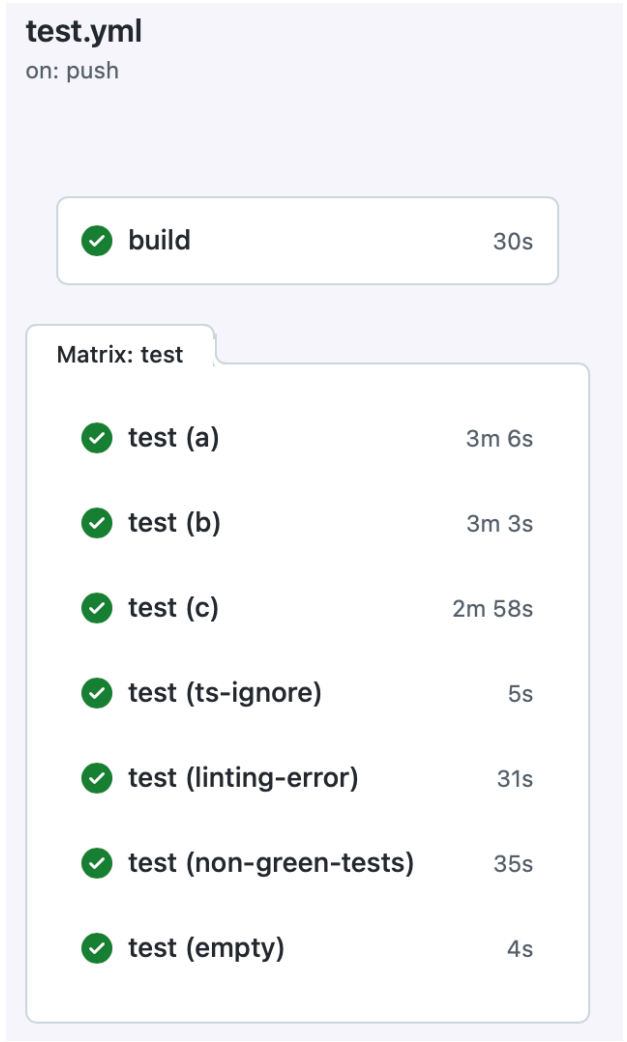


CI In Practice: Autograder from past semester (current version is similar)

test.yml (CI workflow file)

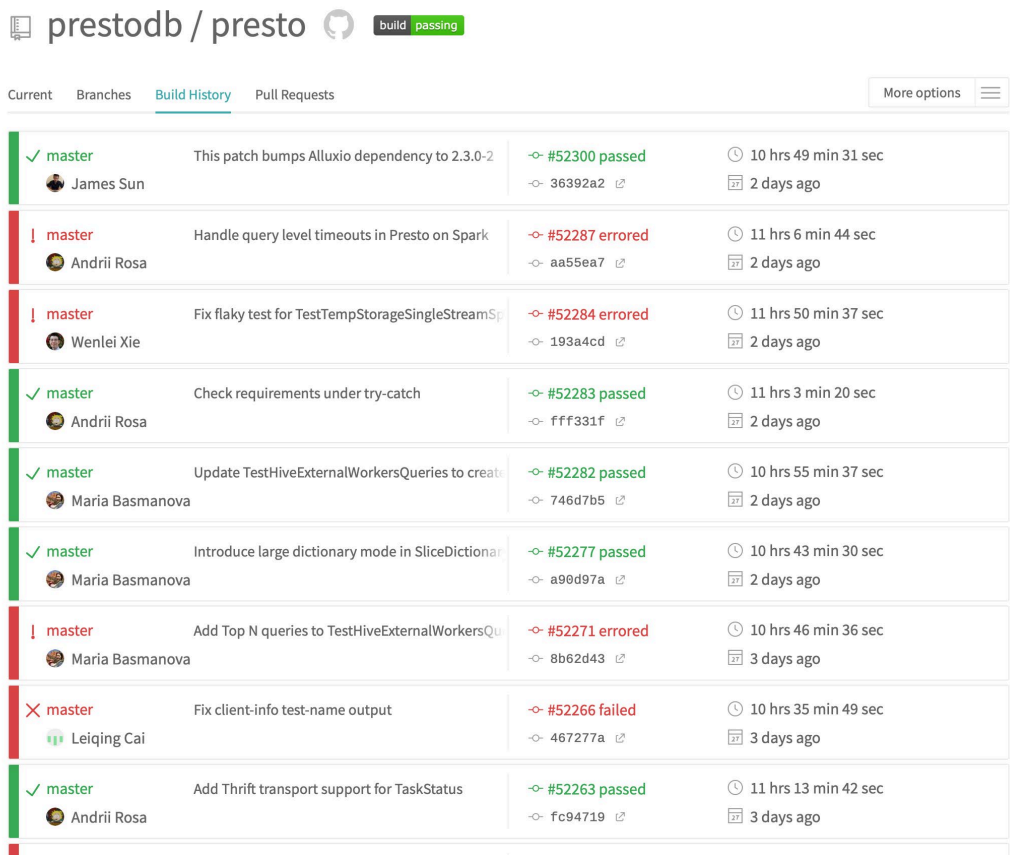
```yaml
name: 'Build and Test the Grader'
on: # rebuild any PRs and main branch changes
  pull_request:
  push:
    branches:
      - main
      - 'releases/*'
jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
        with:
          node-version: '16'
      - run: |
          npm install
  test:
    runs-on: self-hosted
    strategy:
      matrix:
        submission: [a, b, c, ts-ignore, linting-error, non-green-tests, empty]
    steps:
      - uses: actions/checkout@v2
      - uses: actions/setup-node@v2
        with:
          node-version: '16'
      - uses: ./
        with:
          submission-directory: solutions/${{ matrix.submission }}
```

GitHub Actions Results

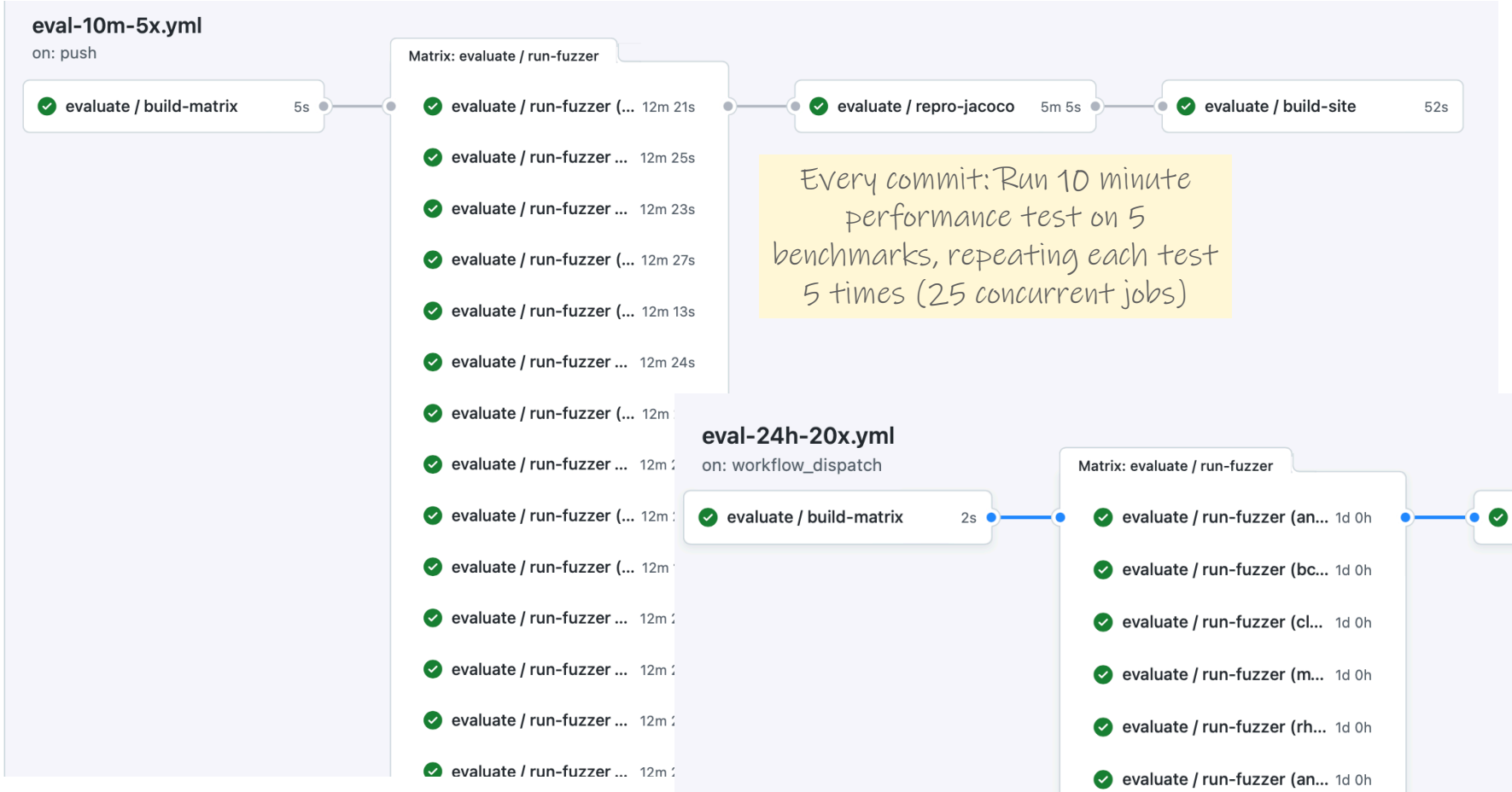


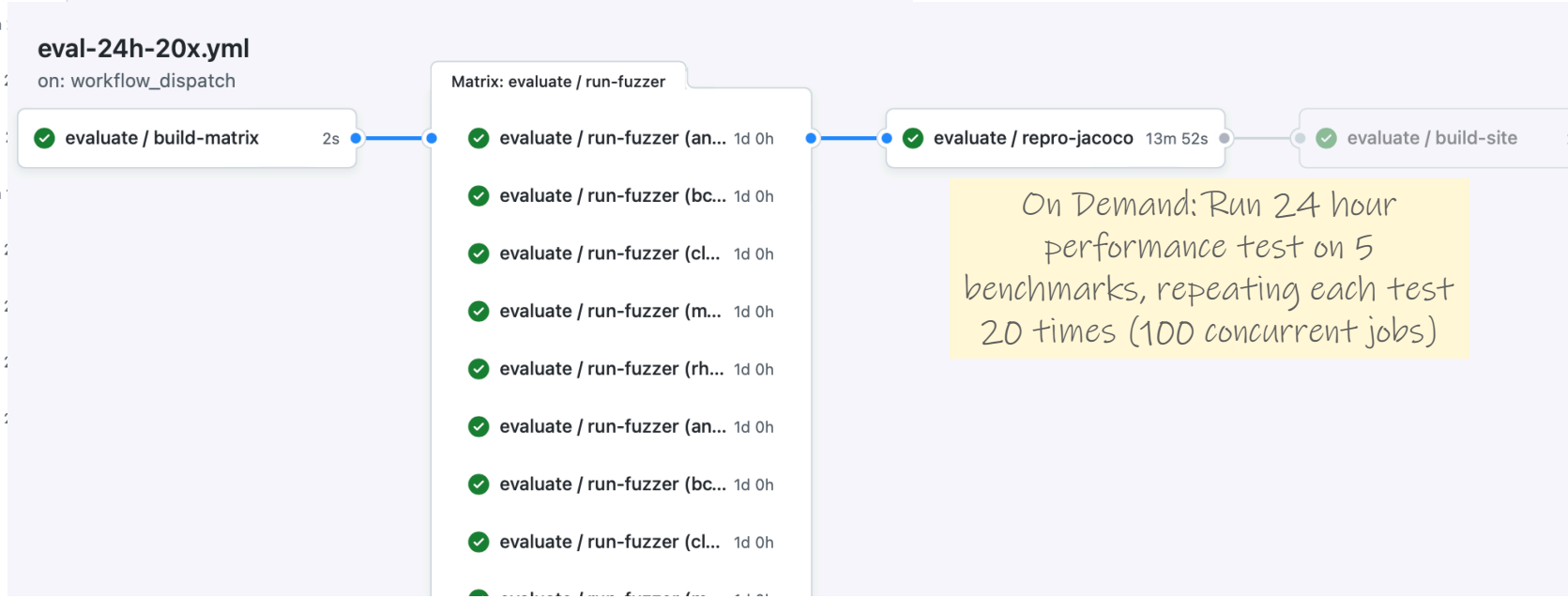
CI processes are run often enough to reduce debugging effort

- Failed CI runs indicate a bug was introduced, and caught in that run
- More changes per-CI run require more manual debugging effort to assign blame
- A single change per-CI run pinpoints the culprit



CI pipelines can automate performance testing



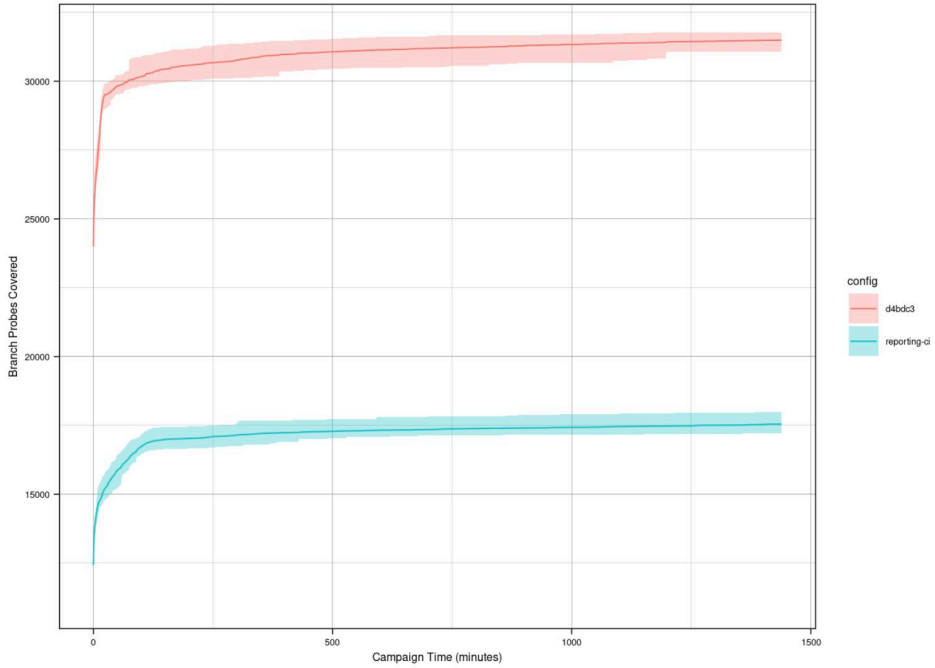


https://github.com/neu-se/CONFETTI/actions

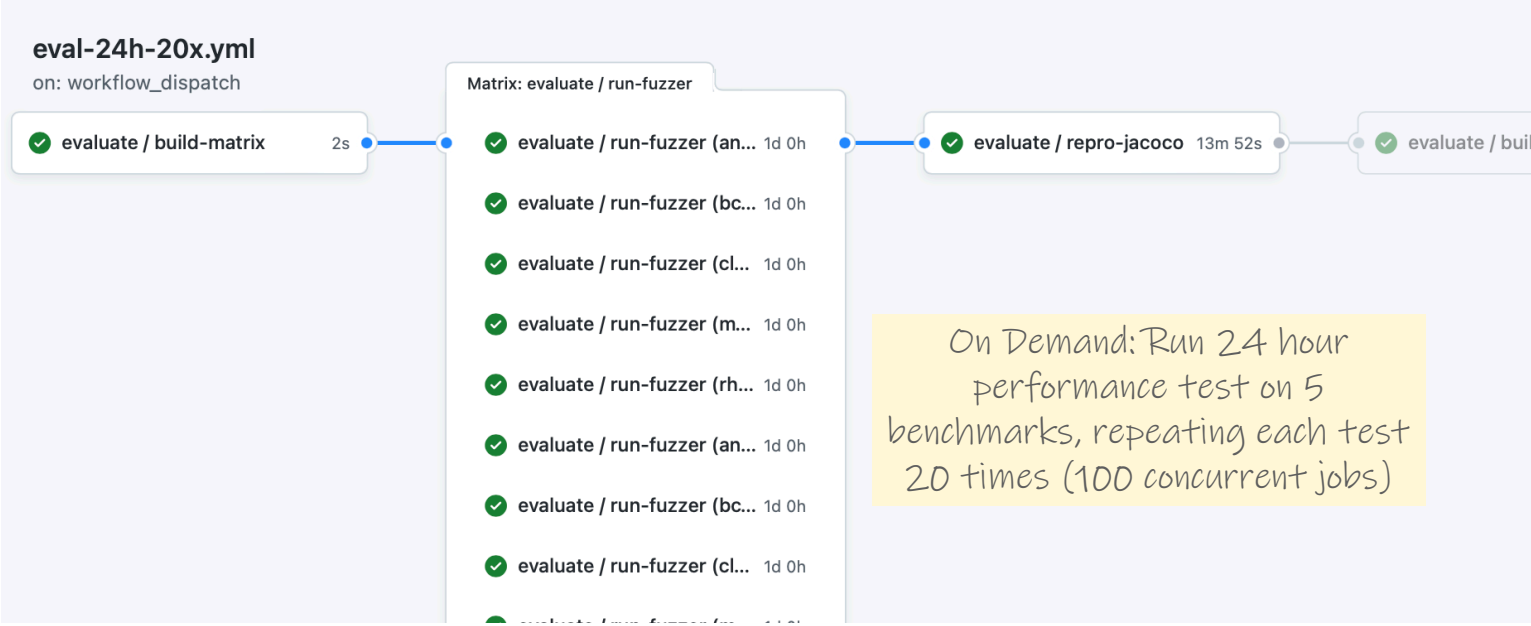
CI pipelines can automate benchmarking

closure

Branch Probes Over Time



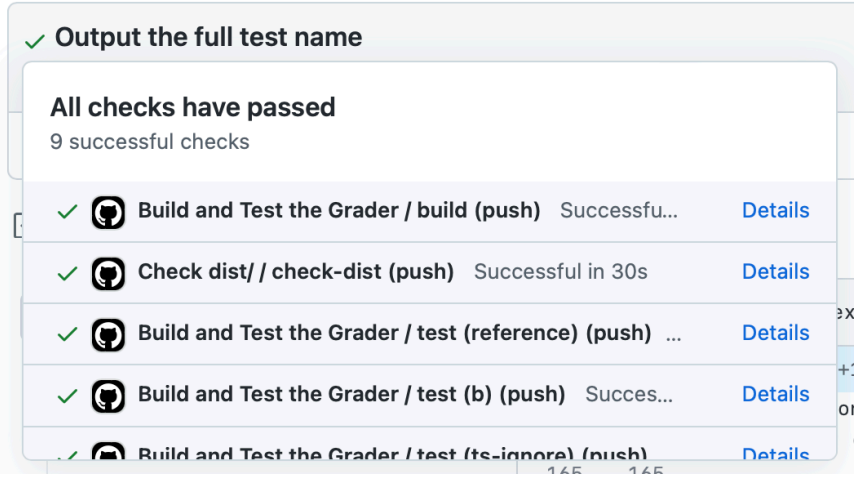
Download this graph as PDF

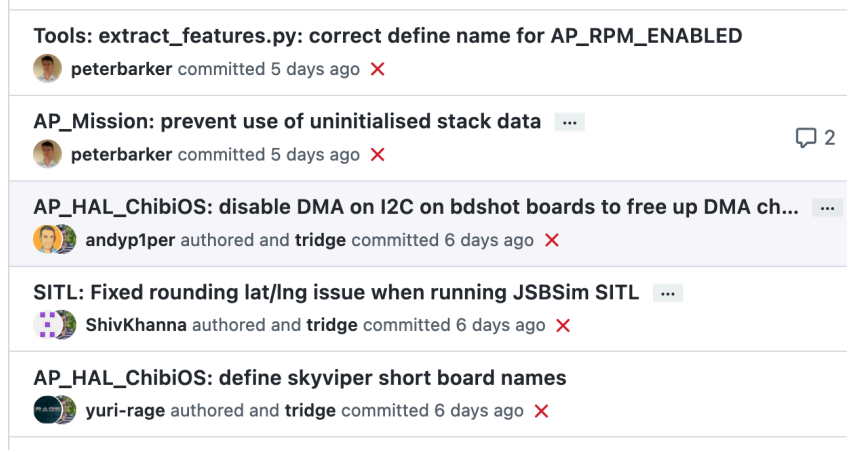


https://github.com/neu-se/CONFETTI/actions

Attributes of effective CI processes

- Policies:
  - Do not allow builds to remain broken for a long time
  - CI should run for every change
  - CI should not completely replace pre-commit testing
- Infrastructure:
  - CI should be fast, providing feedback within minutes or hours
  - CI should be repeatable (deterministic)
  - CI processes should allocate enough resources to mitigate flaky tests





Build Systems Orchestrate Software Engineering Tasks

- “Orchestrate” -> Execute in the right order, ideally with concurrency, example tasks:
  - Installing dependencies
  - Compiling the code
  - Running static analysis
  - Generating documentation
  - Running tests
  - Creating artifacts for customers
  - Deploying Code
- Example build systems: xMake, ant, maven, gradle, npm…

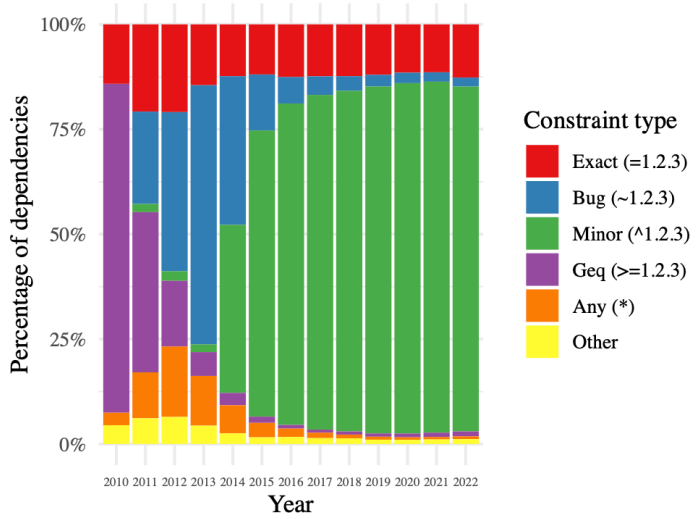
- In most modern languages, the build system itself also serves as the dependency manager

Dependency Managers Organize External Dependencies

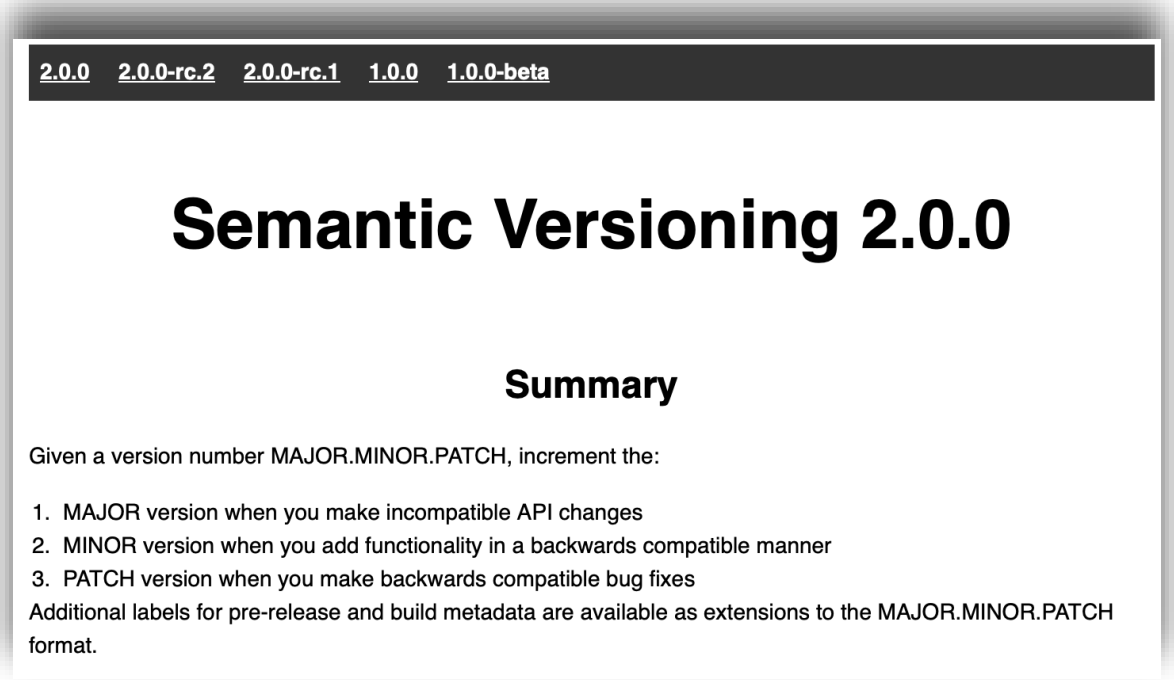
- Addresses this problem: “Before you compile this code, install commons-lang from the Apache website”
- Declare a dependency using coordinates (unique ID of a package plus version)
- Packages are archived in common repositories; fetched/linked by dependency manager
- Dependency managers handle transitive dependencies 
- Examples: Maven, NPM, pip, cargo, apt

Specify and Depend on Package Versions with Care

- Semantic Versioning is often expected:
  - Library maintainers expected to indicate breaking changes with version numbers
  - Dependency consumers can specify constraints on versions (e.g. accept 2.0.x)



Distribution of dependencies of all packages in NPM over time (2023, Pinckney et al)



Continuous Integration Service Models

- Self-hosted/managed on-premises or in cloud
  - Jenkins
- Fully cloud managed
  - GitHub Actions, CircleCI, Travis, many more…
  - Billing model: pay per-build-minute running on SaaS infrastructure
  - “Self-hosted runners” run builds on your own infrastructure, usually “free”

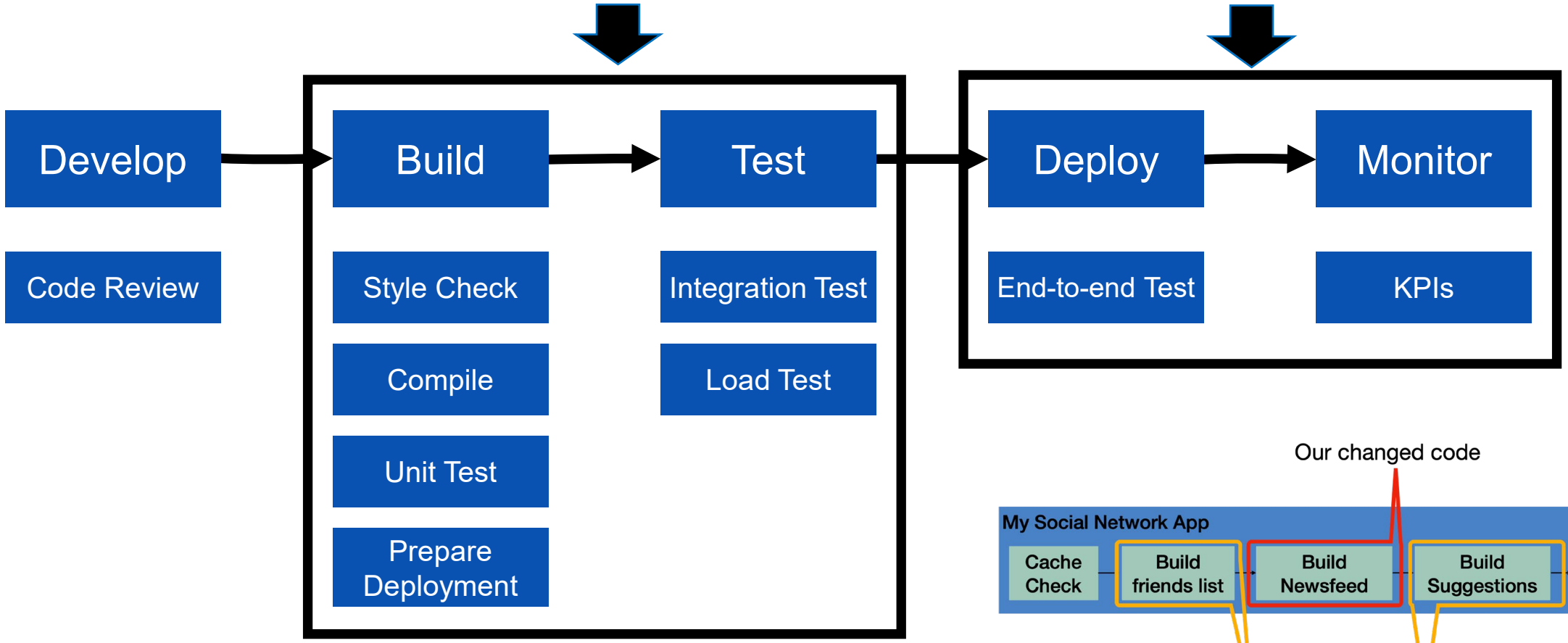
12.2 Continuous Delivery

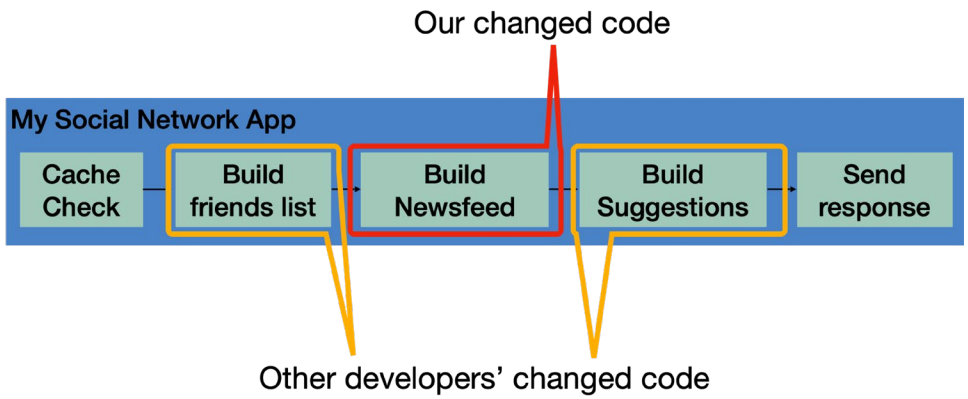
Continuous Delivery is about deciding which new features to deliver, and when

- “Faster is safer”: Key values of continuous delivery
  - Release frequently, in small batches
  - Maintain key performance indicators to evaluate the impact of updates
  - Phase roll-outs
  - Evaluate business impact of new features

A continuous-delivery process is also a software pipeline

Automate this centrally, provide a central record of results





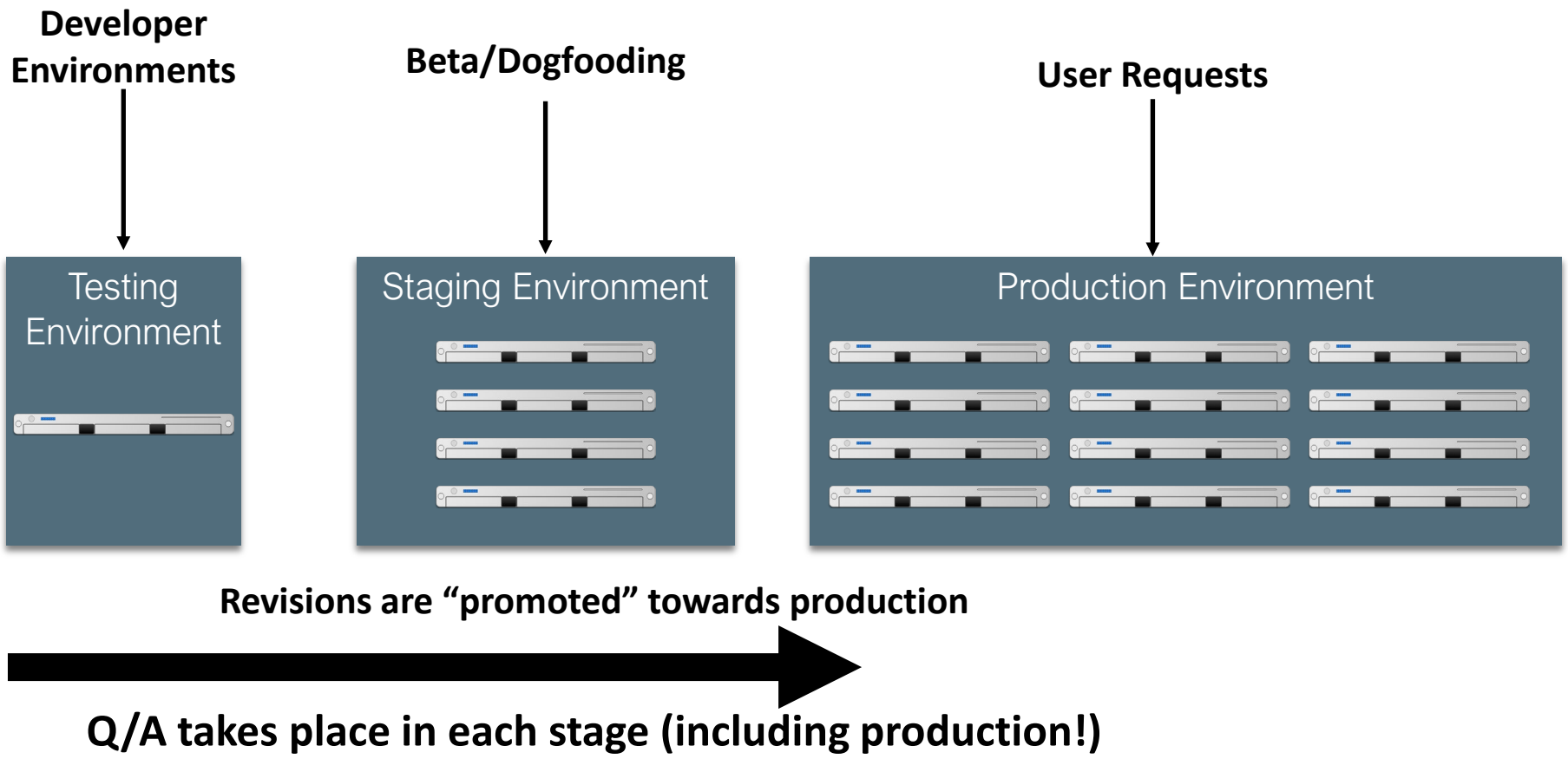
Continuous Delivery does not mean Immediate Delivery

- Even if you are deploying every day (“continuously”), you still have some latency
- A new feature I develop today won't be released today
- But, a new feature I develop today can begin the **release pipeline** today (minimizes risk)
- **Release Engineer**: gatekeeper who decides when something is ready to go out, oversees the actual deployment process

Ways to mitigate deployment risks

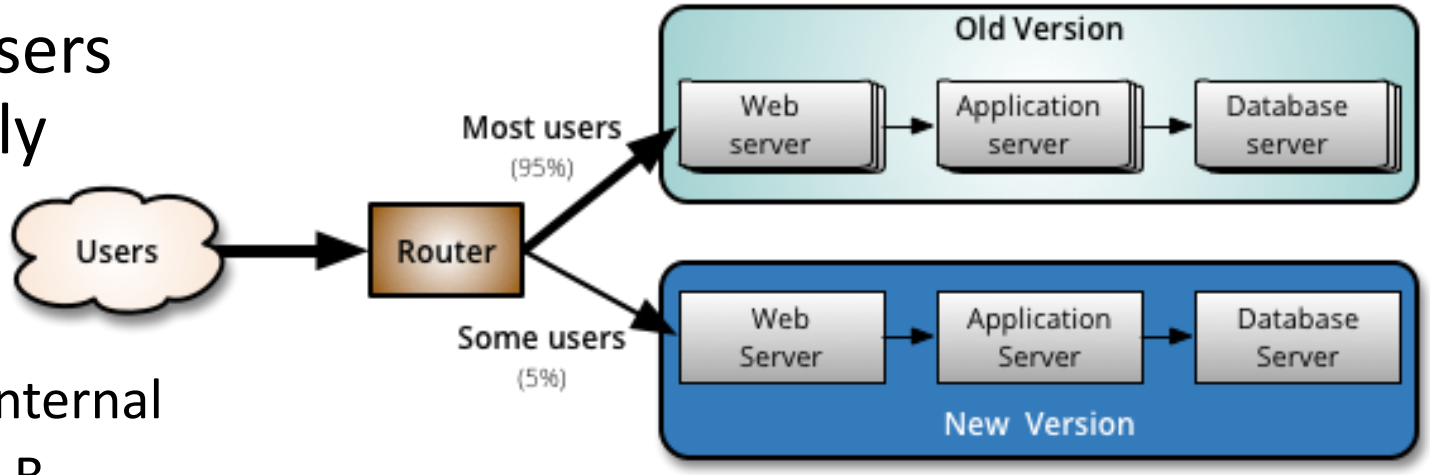
- Use a realistic staging environment
- Use split deployments
- Use post-deployment monitoring
- Use tools to automate deployment tasks

Build a staging environment to qualify features for delivery



Split Deployments Mitigate Risk

- Lower risk if a problem occurs in staging than in production
- Or deploy to a small set of users before deploying more widely
- Names:
  - “Eat your own dogfood”
  - Beta/Alpha testers: external vs internal
  - A/B testing: version A vs version B
  - "canaries“

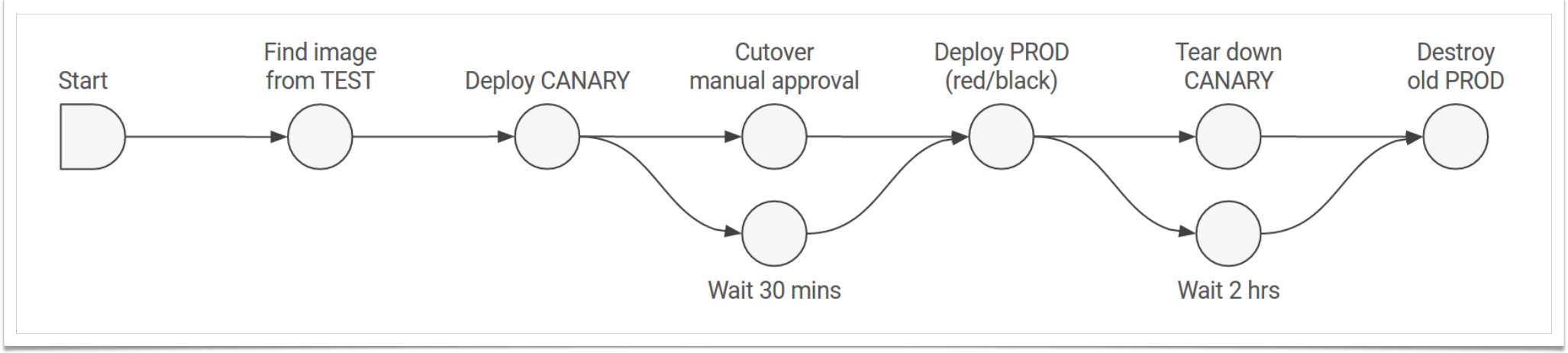


Post-delivery monitoring mitigates risk

- Consider both direct (e.g. business) metrics, and indirect (e.g. system) metrics
  - Hardware
  - Voltages, temperatures, fan speeds, component health
  - OS
  - Memory usage, swap usage, disk space, CPU load
  - Middleware
  - Memory, thread/db connection pools, connections, response time
  - Applications
  - Business transactions, conversion rate, status of 3rd party components

Continuous Delivery Tools

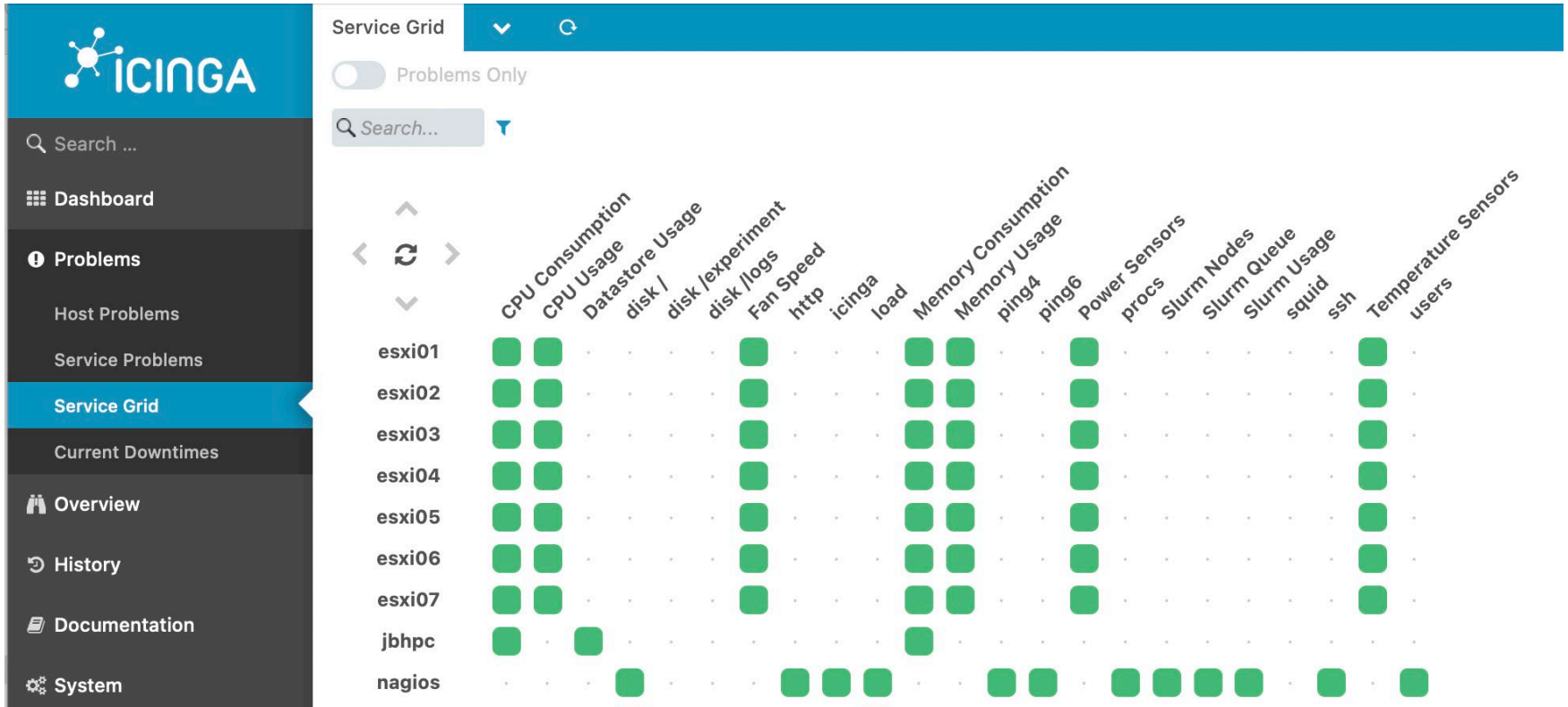
- Simplest tools deploy from a branch to a service (e.g. Render.com, Heroku)
- More complex tools:
  - Auto-deploys from version control to a staging environment + promotes through release pipeline
  - Monitors key performance indicators to automatically take corrective actions
  - Example: "Spinnaker" (Open-Sourced by Netflix, c 2015)



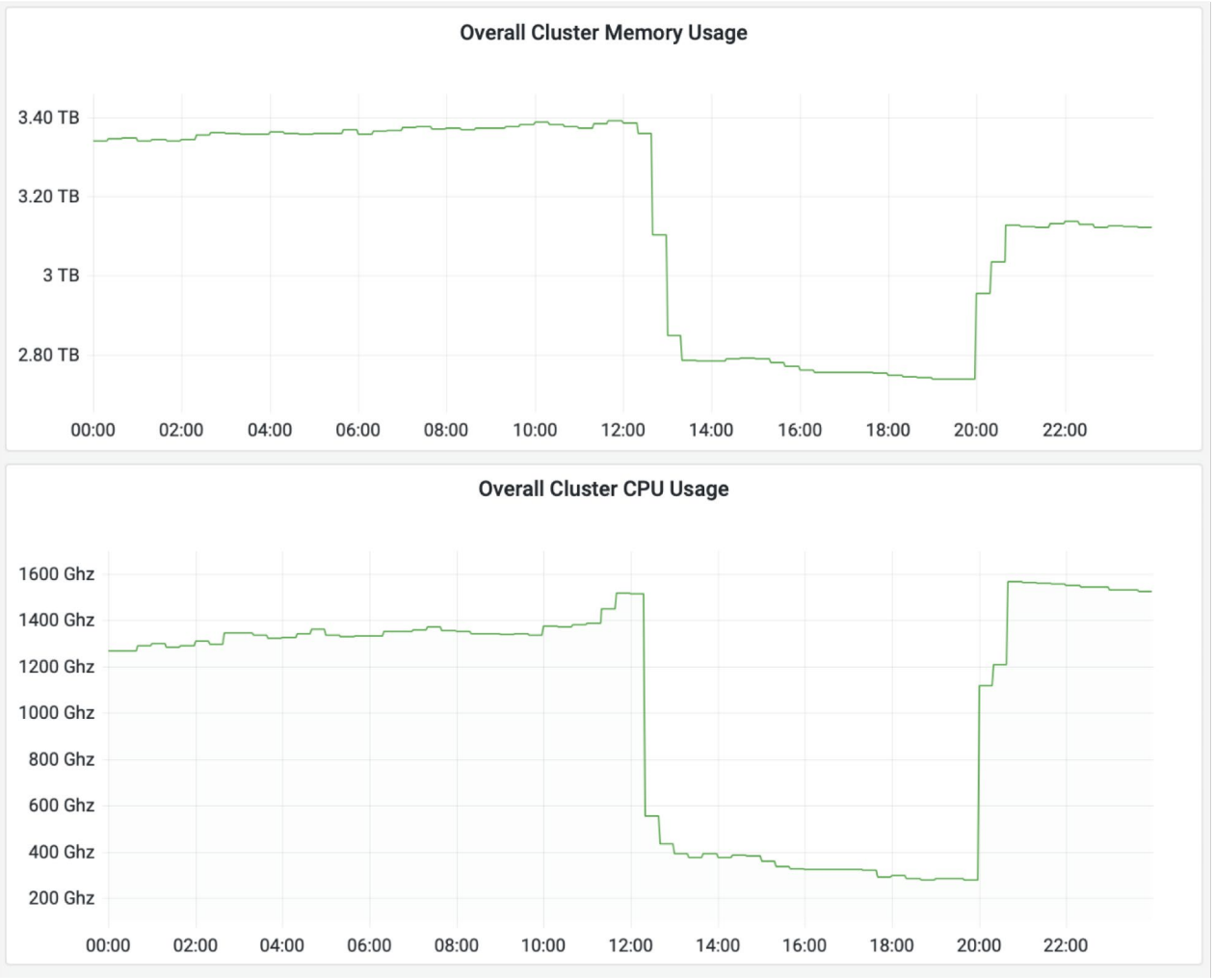
Example CD pipeline from Spinnaker's documentation: https://spinnaker.io/docs/concepts/#application-deployment

Tools for Monitoring Deployments

- Nagios (c 2002): Agent-based architecture (install agent on each monitored host), extensible plugins for executing “checks” on hosts
- Track system-level metrics, app-level metrics, user-level KPIs



Monitoring can help identify operational issues



Grafana (AGPL, c 2014)



InfluxDB (MIT license, c 2013)

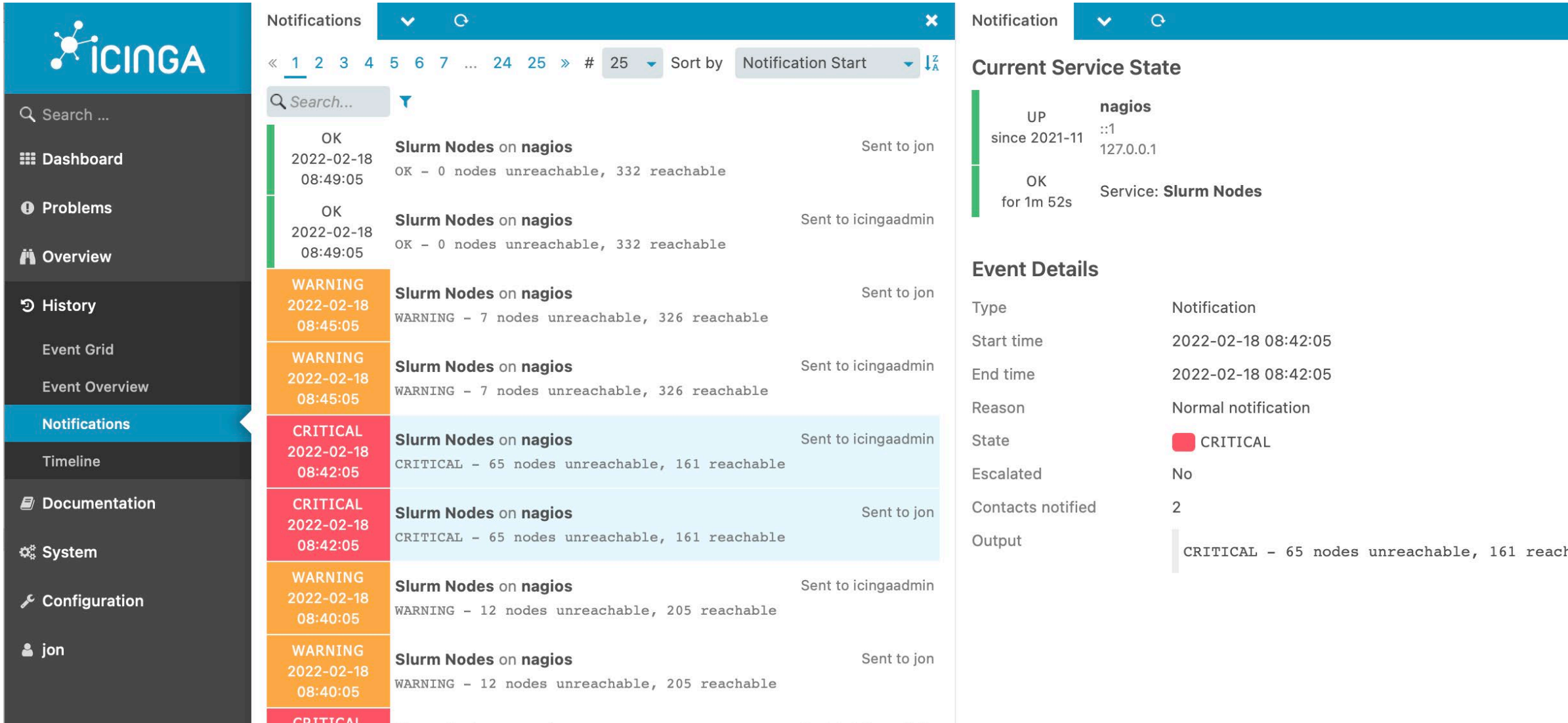
Continuous Delivery Tools Can Take Automated Actions

- Example: Automated roll-back of updates at Netflix based on "streams-per-second" (SPS)



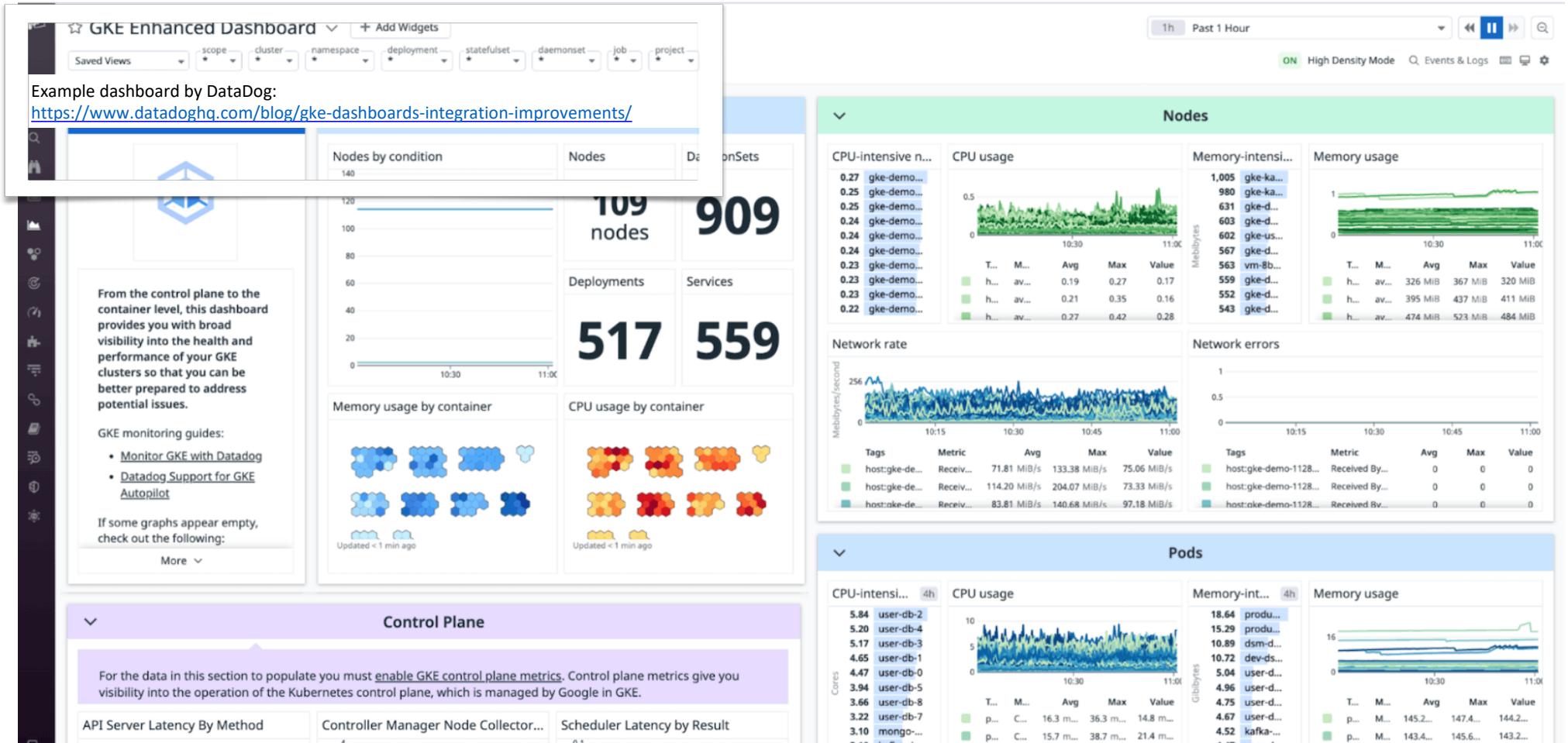
https://www.youtube.com/watch?v=qyzymLlj9ag

Monitoring Services Can Take Automated Actions



From Monitoring to Observability

- Understanding what is going on inside of our deployed systems by visualizing internal metrics



Beware of Metrics

- McNamara Fallacy
  - Measure whatever can be easily measured
  - Disregard that which cannot be measured easily
  - Presume that which cannot be measured easily is not important
  - Presume that which cannot be measured easily does not exist



How should we allocate our testing resources? (For GameNite)

- How much unit testing should be required?
- When should we do code reviews?
- How often should we do integration tests?
- Different organizations may make different choices

Continuous Delivery works with or without TDD

- Test driven development ("Test first") relies on larger test suites
  - Write and maintain tests per-feature (manual! hard!)
  - Unit tests help locate bugs (at unit level)
  - Integration/system tests also needed to locate interaction-related faults
- Continuous delivery work with smaller test suites
  - Write and maintain high-level observability metrics
  - Deploy features one-at-a-time, look for canaries in metrics
  - Write fewer integration/system tests

CI at scale: Google Test Automation Platform - TAP (2020)

- Massive continuous build of entire Google codebase
  - in a dedicated data center
  - 50,000 unique changes per-day, 4 billion test cases per-day
- Engineers submit unit tests along with their changes
  - Block merge if they fail
- If they pass, change is put in the codebase.
  - visible to entire company!
  - average wait time to this point: 11 minutes
- Then (asynchronously) run all affected integration tests
  - If any fail, change is sent back to a human on the submitter's team (the “build cop”) who must act immediately to roll-back or fix.

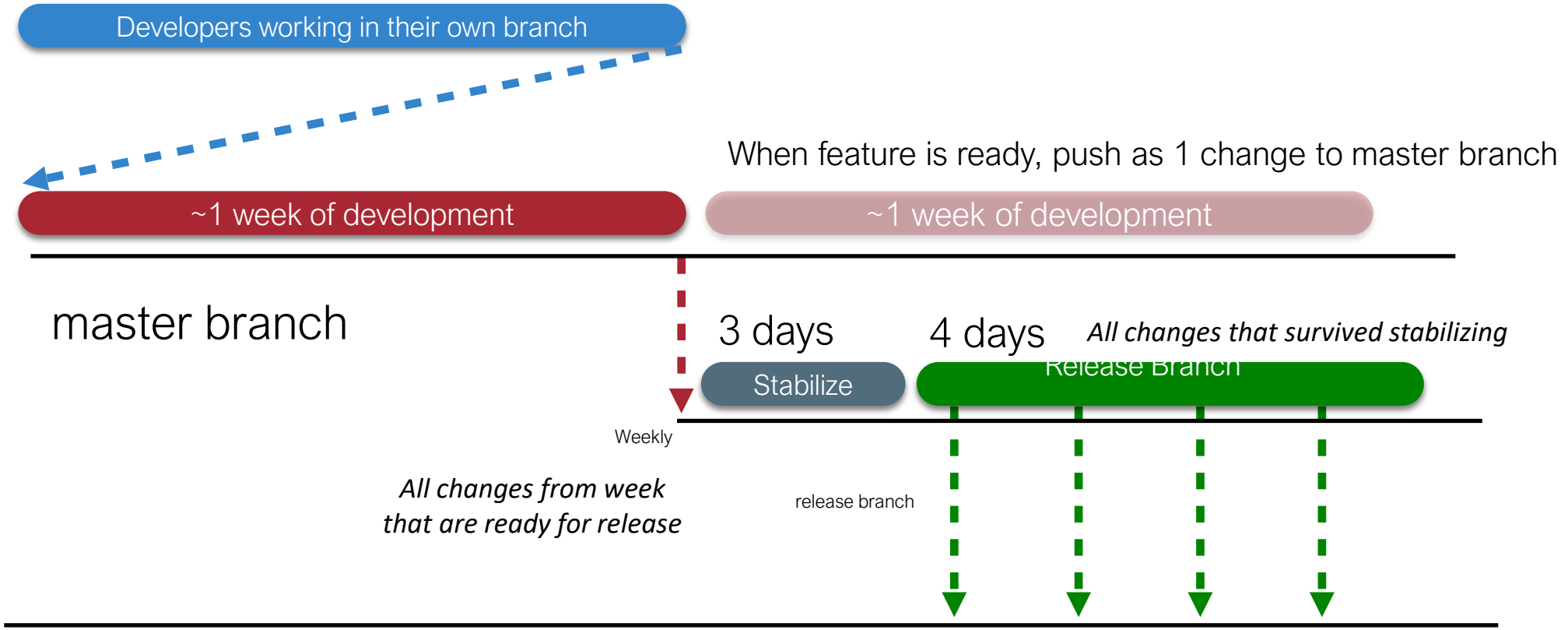
“Software Engineering at Google: Lessons Learned from Programming Over Time,” Wright, Winters and Manshreck, 2020 (O’Reilly), pp. 494-497

Facebook: "Move fast and break things"

- de-prioritize unit tests
- Emphasis on getting features to users quickly
- Strategy: push many small changes to fractions of the user base. ("split deployments")

Deployment Example: Facebook.com

- Pre-2016



Facebook used to have an elaborate system of branches

- dev branches got merged into master,
- then once a week all changes from the past week were pulled into a release branch (often 10,000 changes per week)
- For 3 days they “stabilized” the release branch – find changes that are causing very bad behavior and back them out. (manual process!!)
- Then for the last 4 days of the week, every change that survived that stabilization got *individually pushed* to production batched so that this happens 3x/day.
- Important to do small deploys so that you could isolate bad changes.

Deployment Example

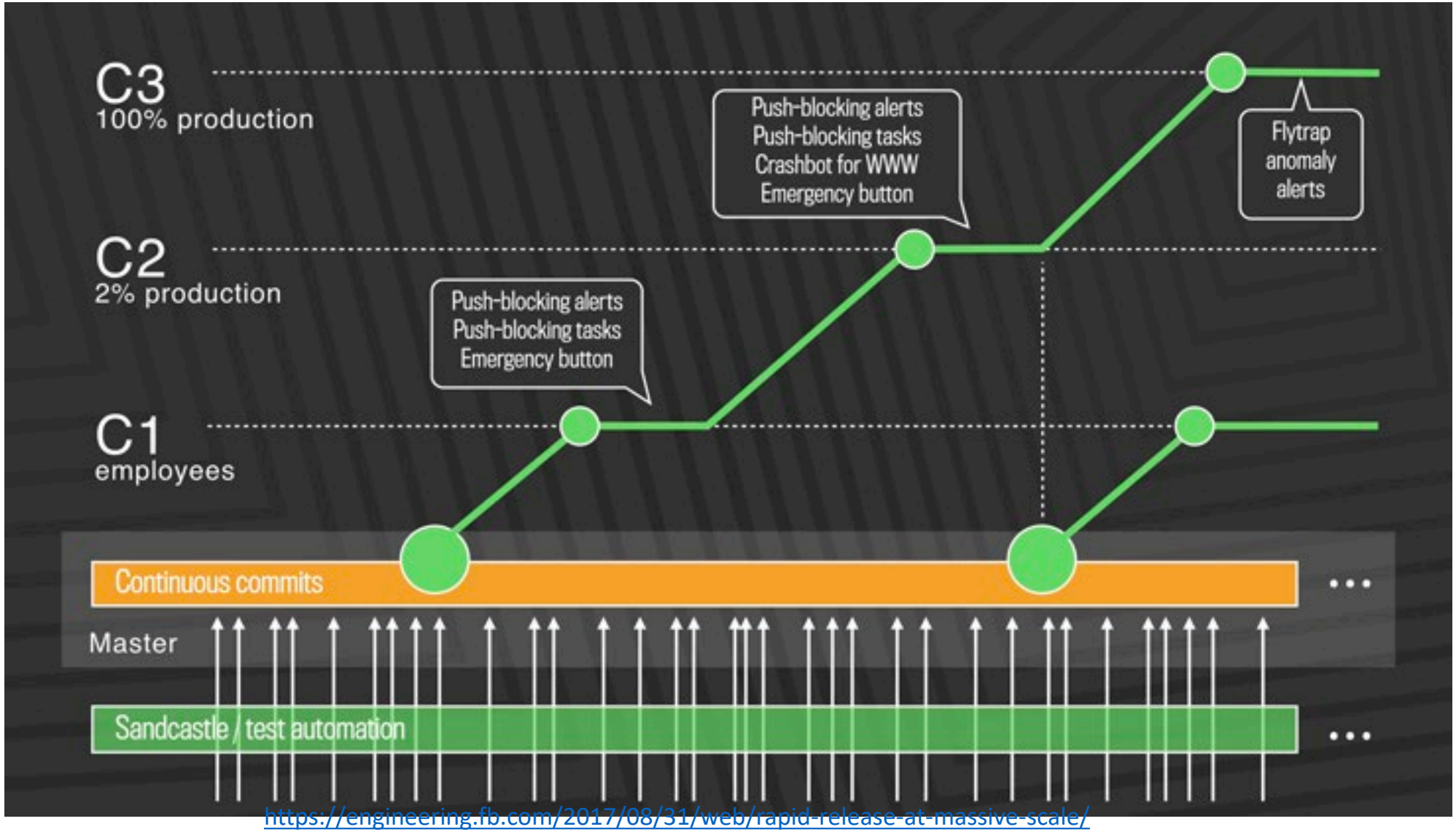


"Our main goal was to make sure that the new system made people's experience better — or at least, didn't make it worse. After a year of planning and development, over the course of three days **we enabled 100% of our production web servers to run code deployed directly from master**"

- Chuck Rossi, Director Software Infrastructure & Release Engineering @ Facebook

"Rapid release at massive scale" https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/

Post-2016: truly continuous releases from master branch



https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/

Post-2016: Truly Continuous Releases from Master Branch (excerpts from blog post)

1. First, diffs that have passed a series of automated internal tests and land in master are pushed out to Facebook employees.
2. In this stage, get push-blocking alerts if we've introduced a regression, and an emergency stop button lets us keep the release from going any further.
3. If everything is OK, push the changes to 2 percent of production, where again we collect signal and monitor alerts, especially for edge cases that our testing or employee dogfooding may not have picked up.
4. Finally, roll out to 100 percent of production, where our Flytrap tool aggregates user reports and alerts us to any anomalies.
5. Many of the changes are initially kept behind feature flags, which allows to roll out mobile and web code releases independently from new features, helping to lower the risk of any particular update causing a problem.
6. If we do find a problem, simply switch the feature off rather than revert back to a previous version or fix forward.

https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/

What not to do: Failed Deployment at Knight Capital

"In the week before go-live, a Knight engineer manually deployed the new RLP code in SMARS to its 8 servers. However, he made a mistake and did not copy the new code to one of the servers. Knight did not have a second engineer review the deployment, and neither was there an automated system to alert anyone to the discrepancy. "

Knightmare: A DevOps Cautionary Tale

D7 DevOps April 17, 2014 6 Minutes

I was speaking at a conference last year on the topics of DevOps, Configuration as Code, and Continuous Delivery and used the following story to demonstrate the importance making deployments fully automated and repeatable as part of a DevOps/Continuous Delivery initiative. Since that conference I have been asked by several people to share the story through my blog. This story is true – this really happened. This is my telling of the story based on what I have read (I was not involved in this).

This is the story of how a company with nearly $400 million in assets went bankrupt in 45-minutes because of a failed deployment.



https://www.henricodolfing.com/2019/06/project-failure-case-study-knight-capital.html

What could Knight capital have done better?

- Use capture/replay testing instead of driving market conditions in a test
- Avoid including “test” code in production deployments
- Automate deployments
- Define and monitor risk-based KPIs
- Create checklists for responding to incidents

Review

- By now, you should be able to…
  - Describe how continuous integration helps to catch errors sooner in the software lifecycle
  - Describe strategies for performing quality-assurance on software as and after it is delivered
  - Understand how continuous delivery can work with or without TDD (test driven development) as a quality assurance strategy